

Examen Parcial 1 — 2023

Sistemas Operativos
Licenciatura en Ciencias de la Computación

Tomás Rodeghiero
UNRC — Universidad Nacional de Río Cuarto

2023

Resumen

Resolución completa del Parcial 1 del año 2023 de la cátedra de Sistemas Operativos (UNRC, Lic. en Ciencias de la Computación). El parcial consta de diez ejercicios que recorren los temas fundamentales del primer cuatrimestre: seguridad y modos de privilegio, contraste entre kernel monolítico y microkernel, instrucciones privilegiadas, mecánica de las syscalls, secuencia de `fork/exec/wait` en un shell, bloqueo del proceso ante una syscall lenta como `read`, transición entre estados (`wakeup`), *starvation* en distintos algoritmos de planificación, *race conditions* en accesos a un saldo bancario compartido, y las limitaciones de spinlocks en monoprocesador y de la inhibición de interrupciones en multiprocesador. Cada ejercicio se desarrolla con la estructura **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

Índice

1. Ejercicio 1 — Seguridad y modos de privilegio	1
2. Ejercicio 2 — ¿Monolítico es más eficiente que microkernel?	2
3. Ejercicio 3 — Instrucciones privilegiadas	3
4. Ejercicio 4 — Propósito y mecanismo de las syscalls	5
5. Ejercicio 5 — Shell ejecutando <code>cmd1 & cmd2</code>	6
6. Ejercicio 6 — ¿ <code>read</code> hace busy-wait?	8
7. Ejercicio 7 — ¿ <code>wakeup</code> pasa procesos a RUNNING?	9
8. Ejercicio 8 — ¿Qué algoritmos generan starvation?	9
9. Ejercicio 9 — Race condition en sistema bancario	11
10. Ejercicio 10 — Spinlocks en uniproc y deshabilitar IRQs en multiproc	13
Resumen del parcial	15

1. Ejercicio 1 — Seguridad y modos de privilegio

Enunciado. Un sistema de multi-programación o multi-tarea debe compartir recursos.

- (a) ¿Qué problemas de seguridad surgen?
- (b) ¿Es posible lograr un sistema seguro con una CPU con un solo modo de privilegio?

1.a — Problemas de seguridad al compartir recursos

Cuando varios procesos comparten CPU, memoria, dispositivos y archivos, surgen riesgos clásicos:

- **Confidencialidad de la memoria:** sin protección, un proceso podría leer la memoria de otro y obtener datos sensibles (contraseñas, claves, datos privados).
- **Integridad de la memoria:** un proceso podría *modificar* la memoria de otro (incluido el código del kernel), corrompiéndolo o inyectando código malicioso.
- **Acceso no autorizado a dispositivos:** lectura directa del disco/red *bypaseando* los permisos del FS; manipulación del audio o pantalla de otro usuario; etc.
- **Denegación de servicio (DoS):** un proceso podría monopolizar la CPU (no ceder voluntariamente), acaparar toda la RAM, llenar el FS, o bloquear dispositivos compartidos.
- **Acceso no autorizado a archivos:** lectura/escritura de archivos de otros usuarios sin permiso.
- **Ejecución de instrucciones críticas:** si un proceso pudiera ejecutar instrucciones como “modificar tabla de páginas” o “deshabilitar interrupciones”, podría secuestrar el sistema entero.
- **Escalada de privilegios:** un proceso de usuario común podría hacerse pasar por root o por otro usuario.

1.b — ¿Sistema seguro con un solo modo de privilegio?

Respuesta

No, en general. Sin distinción entre *modo usuario* y *modo kernel*, cualquier programa puede ejecutar todas las instrucciones de la CPU, incluidas las críticas:

- Modificar la tabla de páginas $\Rightarrow$ acceder a memoria de otros procesos o del kernel.
- Deshabilitar las interrupciones $\Rightarrow$ tomar la CPU para siempre.
- Tocar los registros de los dispositivos $\Rightarrow$ manipular hardware directamente.

Sin un modo privilegiado que el SO use exclusivamente, no hay forma de *forzar* que un programa pase por las syscalls y los chequeos correspondientes. Por eso **toda CPU moderna tiene al menos dos modos** (user / kernel) y muchas tienen más (x86: 4 rings; ARM: 4 niveles de excepción; RISC-V: M, S, U).

Excepciones marginales: sistemas embebidos con software totalmente confiable y auditado (un controlador empotrado de un único programa, p. ej.) pueden vivir con un solo modo, pero no son SOs multi-usuario en sentido estricto.

2. Ejercicio 2 — ¿Monolítico es más eficiente que microkernel?

Enunciado. Un diseño monolítico ofrece generalmente mayor eficiencia que un microkernel. ¿Es válida esta afirmación? Justificar.

Análisis

En un **kernel monolítico** (Linux clásico, Windows NT, *xv6*), todos los subsistemas —scheduler, gestor de memoria, FS, drivers, networking— están compilados en un único binario y corren en **modo kernel**, en el mismo espacio de direcciones. La comunicación entre ellos es por *llamadas a función directa* y acceso a estructuras globales (tabla de procesos, tabla de archivos abiertos, etc.).

En un **microkernel** (Mach, MINIX, L4, QNX), el núcleo se reduce al mínimo: planificador, IPC, MMU básica. El resto (FS, drivers, networking, incluso el gestor de procesos) corre como **procesos servidores en modo usuario**. Toda operación que involucre múltiples subsistemas pasa por IPC entre procesos.

Por qué el monolítico es más eficiente

- **Sin cambios de modo extra:** en monolítico, la invocación entre subsistemas del kernel es una llamada a función. En microkernel, requiere **traps adicionales** (uno por cada **send/receive**).
- **Sin cambios de contexto extra:** en monolítico, no hay context switch al invocar otro subsistema. En microkernel, sí: el control se transfiere al proceso servidor.
- **Sin copia de datos entre espacios:** en monolítico, un puntero a un buffer se pasa directamente. En microkernel, muchas veces hay que *copiar* los datos al espacio del servidor (el espacio de direcciones es distinto).

Ejemplo numérico: una syscall `getpid()` simple en un monolítico cuesta ~ 1 trap (decenas de nanosegundos). En un microkernel típico: ~ 2 traps + 2 context switches + copia de mensaje (centenas de nanosegundos, hasta microsegundos en implementaciones ingenuas).

Lo que el microkernel ofrece a cambio

A pesar de la menor eficiencia, los microkernels tienen ventajas:

- **Modularidad:** cada servicio es un proceso aislado.
- **Robustez:** un driver que crashea no tira el sistema (el kernel sigue corriendo; se reinicia el servicio).
- **Seguridad:** cada servicio tiene sus propios privilegios; un bug en un driver no compromete todo.
- **Portabilidad:** agregar/quitar servicios sin recompilar el kernel.

Respuesta

La afirmación es **válida**: un kernel monolítico es generalmente más eficiente que un microkernel para la misma carga, porque evita los cambios de modo, los context switches y la copia de datos entre espacios de direcciones que requiere el IPC del microkernel.

A cambio, el microkernel gana en *modularidad, robustez, seguridad y portabilidad*, lo que justifica su uso en escenarios donde esos atributos son críticos (sistemas embebidos certificados, sistemas de tiempo real, sistemas de alta seguridad).

3. Ejercicio 3 — Instrucciones privilegiadas

Enunciado. Determinar cuáles de las siguientes instrucciones deberían ser privilegiadas.

Criterio general

Una instrucción debe ser **privilegiada** si su ejecución por parte de un proceso de usuario podría **comprometer la seguridad o estabilidad del sistema**. La regla mnemónica: si la instrucción puede afectar a *otros procesos* o al *kernel*, debe ser privilegiada.

3.a — Setear el valor del timer

Respuesta

Privilegiada. El timer es el corazón del scheduler. Un proceso que pudiera modificarlo podría:

- Aumentar su quantum efectivo y monopolizar la CPU.
- Detener el timer y bloquear el sistema entero.
- Falsificar la hora del sistema.

3.b — Leer el valor del timer

Respuesta

No privilegiada. Leer la hora actual es una operación inofensiva. Los procesos normales necesitan saber la hora para muchas operaciones (timestamps de archivos, mediciones de rendimiento, etc.). Por eso existen syscalls como `gettimeofday`, `clock_gettime`, `time`: son accesibles desde modo usuario sin riesgo.

3.c — Ejecutar una instrucción de trap

Respuesta

No privilegiada. La instrucción de trap (`syscall`, `ecall`, `svc`) es *exactamente* el mecanismo que los procesos de usuario usan para invocar al kernel. Si fuera privilegiada, ningún programa podría hacer syscalls. Es la entrada canónica a modo kernel, controlada por el hardware: el procesador salta a una dirección fija definida por el kernel, no a una arbitraria.

3.d — Deshabilitar interrupciones

Respuesta

Privilegiada. Si un proceso pudiera deshabilitar IRQs:

- El timer no podría interrumpirlo $\Rightarrow$ monopolizaría la CPU.
 - Los dispositivos de I/O no podrían notificar al kernel.
 - No habría preemption $\Rightarrow$ multitarea imposible.
- Es probablemente la instrucción más peligrosa fuera del control del kernel.

3.e — Modificar registros de protección de memoria

Respuesta

Privilegiada. Los registros que controlan la MMU (puntero a la tabla de páginas, base/-límite de segmentos) definen qué memoria ve cada proceso. Modificarlos permitiría:

- Acceder a la memoria de otros procesos.
 - Acceder al espacio del kernel.
 - Saltarse cualquier mecanismo de aislamiento.
- Es la base misma de la separación entre procesos.

3.f — Acceder a registros de control y estado de dispositivos

Respuesta

Privilegiada. Los registros de los controladores de dispositivo (disco, red, GPU, etc.) viven en regiones de memoria mapeada por hardware (MMIO) o en el espacio de puertos. El acceso directo permitiría:

- Leer datos del disco saltándose los permisos del FS.
- Inyectar paquetes en la red sin pasar por el firewall del SO.
- Provocar fallas del hardware.

Por eso solo el kernel (y sus drivers) tocan estos registros; los procesos de usuario los acceden indirectamente vía syscalls (`read`, `write`, `ioctl`).

Resumen

#	Instrucción	¿Privilegiada?
a)	Setear el valor del timer	Sí
b)	Leer el valor del timer	No
c)	Ejecutar una instrucción de <code>trap</code>	No
d)	Deshabilitar interrupciones	Sí
e)	Modificar registros de protección de memoria	Sí
f)	Acceder a registros de control de dispositivos	Sí

4. Ejercicio 4 — Propósito y mecanismo de las syscalls

Enunciado. ¿Cuál es el propósito de una llamada al sistema y cómo se pueden pasar los parámetros?

Propósito

Una **llamada al sistema** (syscall) es la interfaz controlada por la cual un proceso de usuario solicita un servicio al kernel.

Sirve para que el proceso pueda realizar operaciones que requieren **privilegios** (acceso a archivos, red, dispositivos, manipulación de procesos, etc.) sin tener él mismo esos privilegios:

- El proceso prepara los argumentos en sus registros (o en el stack o en memoria).
- Ejecuta la instrucción de `trap`.
- El hardware cambia a modo supervisor y salta a una dirección fija del kernel.
- El kernel valida los argumentos, ejecuta la operación en nombre del proceso y devuelve el resultado.
- El control vuelve al proceso, que ahora está nuevamente en modo usuario.

La syscall es el **único punto de entrada controlado** al kernel. Esto garantiza que (a) los chequeos de permisos siempre se hagan, (b) los argumentos sean validados, y (c) no se pueda saltar a código arbitrario del kernel.

Cómo se pasan los parámetros

Hay tres mecanismos clásicos, todos definidos por la *ABI* (*Application Binary Interface*) de cada arquitectura:

1) En registros (la forma estándar moderna)

Los argumentos se colocan en registros designados *antes* de la instrucción de trap, y el número de syscall en otro registro.

Arquitectura	Registros de args. (en orden)	Registro de syscall #
x86-64 (Linux)	rdi, rsi, rdx, r10, r8, r9	rax
ARM (AArch64)	x0-x5	x8
RISC-V	a0-a5	a7

Valor de retorno: usualmente en el mismo registro del primer argumento (rax, x0, a0). El proceso lee de ahí el resultado cuando reanuda en modo usuario.

Ventajas: rápido (no toca memoria) y simple para syscalls con pocos argumentos (hasta 6 normalmente).

2) En el stack

El proceso empuja los argumentos al stack y ejecuta el trap. El kernel los lee del stack del usuario.

Uso histórico: x86 32-bit (int 0x80). Más lento y requiere validar punteros al stack del usuario.

3) Vía una estructura en memoria

Para syscalls con muchos argumentos o argumentos grandes (e.g., ioctl, readv/writev), se pasa un puntero a una estructura en el espacio del proceso, y el kernel la lee después de validar el puntero.

Validación de los argumentos

Importante

El kernel **nunca debe confiar** en los argumentos del proceso. Antes de usarlos, valida:

- Que los punteros estén en el rango virtual del proceso (no apunten al kernel).
- Que los rangos de tamaño sean razonables (e.g., un count de read no sea negativo o excesivo).
- Que el proceso tenga permiso para la operación (e.g., el FD esté abierto para lectura).

La falta de validación es una de las clases más comunes de bugs en kernel.

Respuesta

Propósito: permitir que un proceso de usuario acceda a servicios privilegiados del kernel (FS, red, dispositivos, gestión de procesos) sin tener esos privilegios él mismo. La syscall es el único punto de entrada controlado al kernel; el hardware (vía la instrucción de trap) garantiza que el control salte a una dirección fija definida por el SO.

Paso de parámetros: vía registros (lo más común y rápido, hasta 6 argumentos), vía stack (histórico) o vía estructura en memoria (para argumentos complejos). El número de syscall se coloca en un registro convenido por la ABI. El kernel devuelve el resultado en el mismo registro del primer argumento.

5. Ejercicio 5 — Shell ejecutando cmd1 & cmd2

Enunciado. Describir los pasos y llamadas al sistema que hace un shell en un comando de la forma cmd1 & cmd2.

Análisis: qué significa &

En bash y la mayoría de shells de UNIX, el operador `&` *separa* comandos (similar a `;`) y marca al comando de la izquierda para ejecutarse en **background**:

- `cmd1 &`: ejecuta `cmd1` en background; el shell no espera.
- `cmd1 & cmd2`: ejecuta `cmd1` en background y luego `cmd2` en foreground (con `wait`).

Importante

No confundir con `cmd1 && cmd2` (con dos ampersands), que es *secuencial-condicional*: ejecuta `cmd2` solo si `cmd1` terminó con código 0.

Secuencia de pasos del shell

1. Parseo de la línea de comando. El shell tokeniza la línea y reconoce el operador `&` como separador con semántica de background para `cmd1`.

2. Ejecución de `cmd1` en background.

- 2.1) `fork()`: el shell crea un proceso hijo P_1 (mismo programa, copia del espacio).
- 2.2) En P_1 (hijo): `exec("cmd1", args)`: reemplaza la imagen del proceso con el binario de `cmd1`.
- 2.3) En el shell (padre): **NO** hace `wait()`; el shell guarda el PID en su tabla de *jobs*.
- 2.4) El shell imprime algo como `[1] 12345` (job number y PID).
- 2.5) El shell continúa procesando la siguiente parte de la línea (`cmd2`).

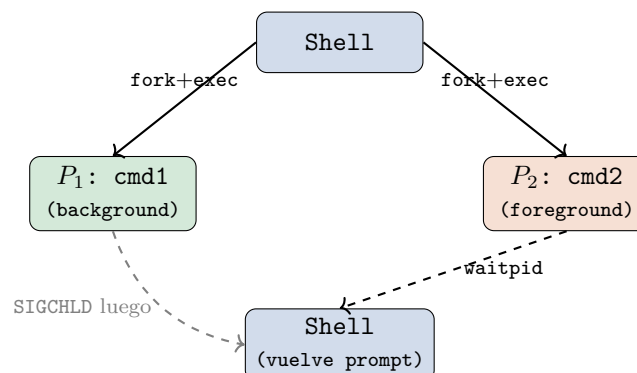
3. Ejecución de `cmd2` en foreground.

- 3.1) `fork()`: el shell crea otro hijo P_2 .
- 3.2) En P_2 : `exec("cmd2", args)`.
- 3.3) En el shell: `waitpid(P2_pid, &status, 0)`: bloquea hasta que `cmd2` termine.
- 3.4) Cuando P_2 termina, el shell vuelve a mostrar el prompt.

4. Cuando `cmd1` (background) eventualmente termina:

- El kernel envía `SIGCHLD` al shell.
- El shell (cuyo handler de `SIGCHLD` llama a `waitpid` en modo no bloqueante `WNOHANG`) recoge el exit status y libera al zombie.
- En la próxima impresión del prompt, el shell notifica al usuario: `[1]+ Done cmd1`.

Diagrama del flujo



Resumen de syscalls usadas

Syscall	Quién la invoca	Función
<code>fork</code>	shell	crear hijo para <code>cmd1</code> , luego otro para <code>cmd2</code>
<code>exec</code>	cada hijo	reemplazar imagen con el binario del comando
<code>waitpid</code>	shell (solo para <code>cmd2</code>)	esperar fin de <code>cmd2</code> (foreground)
<code>waitpid (WNOHANG)</code>	shell en handler de <code>SIGCHLD</code>	recoger exit status de <code>cmd1</code> cuando termine

Respuesta

El shell ejecuta `cmd1 & cmd2` en dos rondas de `fork+exec`: la primera, sin `wait()` (`cmd1` en background, el shell guarda el job y sigue); la segunda, con `waitpid` bloqueante (`cmd2` en foreground). El zombie de `cmd1` se cosecha más tarde vía el handler de `SIGCHLD` o en el próximo prompt.

6. Ejercicio 6 — `£read` hace busy-wait?

Enunciado. En una llamada al sistema `read(fd, buffer, count)` el proceso queda haciendo una espera ocupada hasta que el kernel finalice la operación. Justifique la respuesta.

Análisis

Respuesta

Falso. El proceso **no** hace busy-wait. Pasa al estado `SLEEPING` (bloqueado en una *wait queue*) y libera la CPU. Mientras espera, otros procesos pueden ejecutar.

Por qué no es busy-wait:

El propósito mismo de un SO multitarea es *no desperdiciar CPU*. Si `read` hiciera busy-wait, el proceso quemaría todo su quantum girando sin hacer trabajo útil, mientras la CPU podría estar ejecutando algún otro proceso productivo. Sería el peor diseño posible.

Qué hace el kernel realmente:

1. El proceso ejecuta el trap de `read`.
2. El kernel verifica si los datos ya están en el *buffer cache*:
 - Si SÍ (cache hit): copia al `buffer` del usuario y retorna inmediatamente.
 - Si NO (cache miss): sigue.
3. El kernel envía un comando al controlador del disco para leer el bloque correspondiente.
4. El kernel pone al proceso en una *wait queue* asociada a esa operación: `p->state = SLEEPING`, encola en `wq`.
5. El kernel llama al scheduler ($\rightarrow$ `yield`). Otro proceso obtiene la CPU.
6. Eventualmente, el dispositivo termina la operación y dispara una IRQ.
7. El ISR del driver marca los datos como disponibles y llama a `wakeup(wq)`, que pasa al proceso a `RUNNABLE`.
8. Cuando el scheduler vuelve a elegirlo, el proceso reanuda dentro de `read`, copia los datos al `buffer` de usuario y retorna.

Idea intuitiva

Por qué bloquear en vez de gastar CPU: la I/O de disco es de órdenes de magnitud más lenta que la CPU. Un acceso a un HDD toma ~ 10 ms; en ese tiempo la CPU puede ejecutar $\sim 10^7$ instrucciones útiles. Si el proceso girara en busy-wait, todas esas instrucciones se perderían. El bloqueo es la clave de la *utilización eficiente* de la CPU.

7. Ejercicio 7 — ¿wakeup pasa procesos a RUNNING?

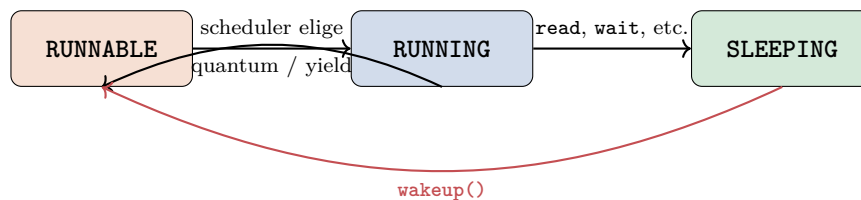
Enunciado. La operación `wakeup()` pasa procesos desde el estado `SLEEPING` a `RUNNING`. Justificar.

Análisis

Respuesta

Falso. `wakeup()` pasa procesos de `SLEEPING` a `RUNNABLE` (también llamado `READY`), **no directamente a `RUNNING`**. Para que un proceso pase a `RUNNING` hace falta que el **scheduler** lo seleccione de la cola `READY` y le asigne CPU.

Diagrama de estados de un proceso



Lo crítico: `wakeup()` NO conecta directamente con `RUNNING`. Va a `RUNNABLE` primero.

Por qué no es directo

- Puede haber muchos procesos `RUNNABLE`; solo uno (o tantos como CPUs haya) puede estar `RUNNING` simultáneamente.
- `wakeup` puede despertar varios procesos a la vez (todos los que esperaban en la misma *wait queue*). No es posible ponerlos a todos en `RUNNING` de inmediato.
- La decisión de quién ejecuta es del scheduler, no del que despertó. El scheduler aplica su política (prioridad, RR, MLFQ, etc.) para elegir.

Excepción: *preemption* al despertar

Algunos kernels (Linux con `CONFIG_PREEMPT`) verifican, después de `wakeup`, si el proceso recién despertado tiene *mayor prioridad* que el actualmente en ejecución. Si es así, disparan inmediatamente un context switch: el despertado entra a `RUNNING` y el actual vuelve a `RUNNABLE`.

Aún en ese caso, la transición lógica es `SLEEPING` $\rightarrow$ `RUNNABLE` $\rightarrow$ `RUNNING` (y la última depende del scheduler, no del `wakeup`).

8. Ejercicio 8 — ¿Qué algoritmos generan starvation?

Enunciado. ¿Cuáles de los siguientes algoritmos de planificación de uso de CPU pueden generar *starvation* (*livelocks*)?

- a) FCFS
- b) Shortest Job/Burst First
- c) Round Robin
- d) Prioridades

Análisis: qué es starvation

Starvation: un proceso espera indefinidamente recibir CPU mientras otros sí la reciben.

Para que haya starvation hace falta que la política de selección *discrimine sistemáticamente* contra algún proceso, sin mecanismo de *aging* que mitigue.

8.a — FCFS

Respuesta

No genera starvation. FCFS es estrictamente por orden de llegada. Cada proceso espera, como mucho, los $n - 1$ procesos que llegaron antes (suma de sus ráfagas). En tiempo finito todos ejecutan.

Aclaración: FCFS sufre el *convoy effect* (procesos cortos esperan detrás de uno largo y aumentan su turnaround promedio), pero eso es *ineficiencia*, no *starvation*: nadie espera infinitamente.

8.b — Shortest Job/Burst First (SJF / SRTF)

Respuesta

SÍ puede generar starvation. SJF siempre elige el proceso con la ráfaga más corta. Si llegan continuamente procesos cortos, los procesos largos *nunca son seleccionados*: cada vez que llegan al frente, hay alguien más corto adelante.

Es el caso paradigmático de starvation en libros de SO. La solución estándar es *aging*: aumentar la prioridad efectiva del proceso en función del tiempo que lleva esperando.

8.c — Round Robin

Respuesta

No genera starvation. RR es estrictamente justo: cada proceso obtiene CPU exactamente una vez cada $\leq N \cdot q$ unidades de tiempo (con N procesos en la cola y quantum q). Ningún proceso queda postergado: el peor caso de espera está acotado.

8.d — Prioridades

Respuesta

SÍ puede generar starvation, si no hay aging. Procesos de baja prioridad pueden quedar indefinidamente postergados si constantemente llegan procesos de mayor prioridad.

La solución estándar (también clásica): *aging*, incrementar la prioridad del proceso a medida que envejece en la cola. Garantiza que tarde o temprano alcanzará la prioridad necesaria para ejecutar.

Resumen

Algoritmo	£Starvation?	Razón / Mitigación
FCFS	No	Estrictamente por orden de llegada
SJF / SRTF	Sí	Procesos largos postergados; <i>aging</i>
Round Robin	No	Justo por construcción; espera acotada
Prioridades (sin aging)	Sí	Baja prioridad postergada; <i>aging</i>

Importante

Starvation vs. livelock. La consigna los menciona juntos pero son distintos:

- **Starvation:** el proceso espera CPU/recurso pero nunca lo obtiene.
- **Livelock:** los procesos sí ejecutan, pero constantemente se ceden el paso o repiten estados sin progresar.

En algoritmos de planificación, el problema relevante es starvation, no livelock (los livelocks típicamente aparecen en protocolos de sincronización mal diseñados).

9. Ejercicio 9 — Race condition en sistema bancario

Enunciado. Un sistema bancario usa archivos para representar el saldo de una cuenta bancaria.

```
deposit(account, amount):
    m = read(account)
    write(account, m + amount)

withdraw(account, amount):
    m = read(account)
    if m > amount:
        write(account, m - amount)
```

- (a) Determinar si existe race condition en un sistema multitarea preemptivo.
- (b) Ofrecer una solución usando algún mecanismo de sincronización.

9.a — £Hay race condition?

Respuesta

Sí, hay race condition de tipo *lost update*. Ambas operaciones son secuencias **read-modify-write** no atómicas sobre el mismo archivo. En un sistema preemptivo, el scheduler puede interrumpir entre el **read** y el **write** de una tarea para correr la otra, produciendo intercalado nocivo.

Traza concreta del bug

Saldo inicial `account` = 100.

Sean dos operaciones concurrentes:

- T_1 : `deposit(account, 30)` (deposita 30).
- T_2 : `withdraw(account, 50)` (retira 50).

Resultado esperado: $100 + 30 - 50 = 80$.

t	T_1 (deposit 30)	T_2 (withdraw 50)	saldo
t_1	$m = \text{read}(\text{account}) \rightarrow 100$		100
t_2	preempt: scheduler cambia a T_2		100
t_3		$m = \text{read}(\text{account}) \rightarrow 100$	100
t_4		if $m > 50$ (verdadero)	100
t_5		$\text{write}(\text{account}, 50)$	50
t_6	preempt: scheduler cambia a T_1		50
t_7	$\text{write}(\text{account}, 100 + 30 = 130)$		130

Resultado real: saldo = 130. Diferencia con el esperado (80): \$50 que aparecieron **de la nada**. Otra traza posible (a la inversa) llevaría a saldo = 50, es decir, \$30 **desaparecen**.

Ambas trazas son posibles y dependen del scheduling. Esto es exactamente el patrón clásico de *lost update*: las dos operaciones leen el mismo valor inicial y se pisan al escribir.

9.b — Solución con un mutex (lock por cuenta)

```
/* Cada cuenta tiene su propio mutex. */
mutex_t account_lock[N_ACCOUNTS];

void deposit(int account, double amount) {
    acquire(&account_lock[account]);
    /* --- region critica --- */
    double m = read(account);
    write(account, m + amount);
    /* --- fin region critica --- */
    release(&account_lock[account]);
}

void withdraw(int account, double amount) {
    acquire(&account_lock[account]);
    /* --- region critica --- */
    double m = read(account);
    if (m > amount) {
        write(account, m - amount);
    }
    /* --- fin region critica --- */
    release(&account_lock[account]);
}
```

Por qué funciona: el mutex hace que el bloque `read` + (cómputo) + `write` sea atómico respecto de cualquier otra invocación sobre *la misma cuenta*. Si dos threads invocan `deposit` y `withdraw` sobre `account`, el segundo en llegar espera hasta que el primero libere el lock; entonces lee el saldo *ya actualizado* y opera correctamente.

Alternativas más robustas

1. **flock() del archivo:** si el saldo está *en un archivo*, `flock(fd, LOCK_EX)` bloquea a otros procesos (no solo threads del mismo proceso) que intenten tomarlo. Esencial cuando el saldo es accedido por *procesos distintos*.
2. **Transferencia entre cuentas:** si hubiera `transfer(from, to)`, hay que tomar *dos* locks. Para evitar deadlocks, imponer **orden global** (tomar primero el lock de la cuenta de menor ID).
3. **Transacciones (estilo BD):** para bancos reales, las operaciones se encapsulan en transacciones *ACID*, gestionadas por un motor de DB. El motor garantiza atomicidad, consistencia, aislamiento y durabilidad.

Idea intuitiva

¿Por qué un lock por cuenta y no un lock global? Si todos los accesos al banco compitieran por el mismo mutex, dos retiros de cuentas distintas se serializarían innecesariamente. Con un lock por cuenta, dos operaciones sobre cuentas distintas *pueden ejecutarse en paralelo*, mejorando el throughput.

10. Ejercicio 10 — Spinlocks en uniproc y deshabilitar IRQs en multiproc

Enunciado. Explicar por qué los spinlocks no son apropiados en sistemas uni-procesadores y por qué deshabilitar interrupciones no alcanza en multi-procesadores.

Parte 1: spinlocks en uniproc

Recordatorio. Un spinlock implementa exclusión mutua con busy-wait:

```
void acquire(lock_t *l) {
    while (atomic_swap(l, 1) == 1)
        ; /* spin */
}
void release(lock_t *l) {
    *l = 0;
}
```

Problema en uniproc: hay *un solo flujo* de ejecución a la vez. Considerá el siguiente escenario:

1. Thread *A* toma el lock *l* y entra a la sección crítica.
2. El timer interrumpe; el scheduler conmuta a thread *B*.
3. *B* ejecuta `acquire(l)`. Como $l = 1$, entra al `while` y *gira*.
4. *B* **quema todo su quantum** girando sin hacer nada útil. La CPU está ocupada 100 % pero el progreso es nulo.

El lock no se libera hasta que *A* vuelva a ejecutar, lo cual no sucederá mientras *B* esté en la CPU. Recién al expirar el quantum de *B*, el scheduler puede planificar a *A*, que termina la SC y libera el lock.

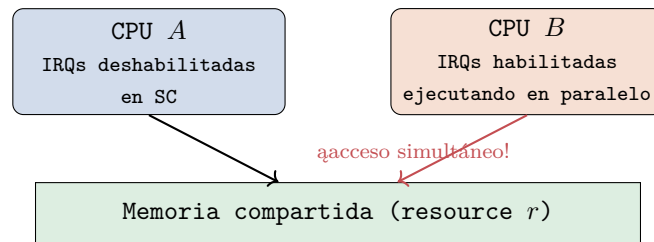
Caso peor: si *B* tiene MAYOR prioridad que *A* y el scheduler nunca lo desaloja, *A* *no vuelve a ejecutar nunca*. *B* gira para siempre. Es una forma de **inversión de prioridad** que produce livelock efectivo.

Conclusión: en uniproc conviene usar *sleep locks* (`pthread_mutex_t`, semáforos): cuando un thread no puede tomar el lock, se **bloquea** (estado `SLEEPING`), liberando la CPU para que otros (incluido el dueño del lock) puedan ejecutar.

Parte 2: deshabilitar IRQs en multiproc

Razonamiento clásico en uniproc: si deshabilito las IRQs, ningún timer ni IRQ de dispositivo puede interrumpirme; nadie me toma la CPU; por lo tanto puedo ejecutar la SC sin que nadie más acceda al recurso compartido. **Esto funciona en uniproc.**

Problema en multiproc: la instrucción `cli` (o equivalente) deshabilita interrupciones sólo en la CPU local.



CPU *A* con sus IRQs deshabilitadas entra a la SC tranquila. Pero CPU *B* sigue ejecutando en paralelo, con sus IRQs habilitadas, y puede tocar la misma estructura sin que nada se lo impida. **La exclusión mutua se rompe.**

Conclusión: deshabilitar IRQs es necesario solo para prevenir la *preemption local*, pero no provee *exclusión entre CPUs*. En multiprocesador hace falta:

1. Una instrucción **atómica** (`xchg`, `cmpxchg`, LL/SC) para coordinar entre las CPUs.
2. Un **spinlock** construido sobre esa instrucción atómica.
3. Opcionalmente, deshabilitar IRQs locales *además* (por ejemplo, si el lock puede ser tomado desde un handler de interrupción de la misma CPU; combinación `spin_lock_irqsave` de Linux).

Síntesis

	Uniproc	Multiproc
Deshabilitar IRQs	Suficiente para EM (pero solo para regiones cortas)	Insuficiente: otras CPUs siguen activas
Spinlock	Mal uso: gasta CPU sin progreso	Apropiado para SC cortas
Sleep lock (mutex)	Apropiado: bloquea sin gastar CPU	Apropiado para SC largas

Respuesta

Spinlocks en uniproc: mal idea. Si un thread tiene el lock y otro lo intenta tomar, el segundo *gira* consumiendo todo su quantum sin progresar; mientras gira, el dueño del lock no puede ejecutarse para liberar. Es desperdicio puro. En uniproc se usan *sleep locks*.

Deshabilitar IRQs en multiproc: insuficiente. Solo afecta a la CPU local; las demás CPUs siguen ejecutando en paralelo y pueden acceder a la misma memoria. Se requieren instrucciones *atómicas* (test-and-set, CAS, LL/SC) y spinlocks construidos sobre ellas.

Resumen del parcial

#	Tema	Respuesta sintética
1	Seguridad y privilegios	(a) problemas: confidencialidad, integridad, DoS, acceso a dispositivos, escalada. (b) No: se necesitan al menos 2 modos
2	Monolítico vs microkernel	V: monolítico más rápido por evitar IPC; microkernel gana en modularidad, robustez y seguridad
3	Instrucciones privilegiadas	a: Sí, b: No, c: No, d: Sí, e: Sí, f: Sí
4	Propósito de syscalls	Acceder a servicios privilegiados controlados; params en registros (ABI), stack o estructura
5	Shell <code>cmd1 & cmd2</code>	<code>fork+exec</code> para <code>cmd1</code> sin <code>wait</code> ; <code>fork+exec+waitpid</code> para <code>cmd2</code> ; <code>SIGCHLD</code> para <code>cmd1</code> al terminar
6	<code>read</code> y busy-wait	F: el proceso pasa a SLEEPING; el SO planifica otros mientras espera la I/O
7	<code>wakeup</code> y RUNNING	F: <code>wakeup</code> pasa SLEEPING → RUNNABLE; el scheduler lo lleva a RUNNING
8	Algoritmos y starvation	FCFS y RR no; SJF y prioridades sí (mitigable con aging)
9	Race condition bancaria	(a) Sí, <i>lost update</i> clásico; (b) mutex por cuenta sobre el bloque read-modify-write
10	Spinlocks y deshabilitar IRQs	Spinlock en uniproc desperdicia CPU; deshabilitar IRQs en multiproc solo afecta la CPU local. Se necesitan instrucciones atómicas